


UNIVERSITETI I EVROPËS JUGLINDORE
УНИВЕРЗИТЕТ НА ЈУГОИСТОЧНА ЕВРОПА
SOUTH EAST EUROPEAN UNIVERSITY

FACULTY OF CONTEMPORARY SCIENCES AND TECHNOLOGIES
UNDERGRADUATE STUDIES IN THE FIELD OF COMPUTER SCIENCE

# MapOut

## Week 4: Uninformed Search and Representation

**Student:** Mirjana Arsenoska
**Student ID:** 131834

**Professor:** Prof. Dr. Azir Aliu
**Course:** Introduction to Artificial Intelligence

Tetovo, 2026

# CONTENT

**1. Problem Representation for MapOut** ..... 1
1.1 Initial State ..... 1
1.2 State Representation ..... 2
1.3 State Space ..... 2
1.4 Actions ..... 3
1.5 Transition Model ..... 3
1.6 Goal Test ..... 4
1.7 Path Cost ..... 4
**2. Uninformed Search Algorithms** ..... 5
2.1 Breadth-First Search (BFS) ..... 5
2.2 Depth-First Search (DFS) ..... 5
2.3 Example of BFS and DFS in MapOut ..... 6
2.4 Search Strategy for MapOut ..... 7
**3. Data and Environment Preparation** ..... 8
3.1 Initial Dataset Structure ..... 8
3.2 Why I Chose a Small Dataset ..... 9
3.3 Data Quality and Limitations ..... 9
**Conclusion** ..... 10
**AI Use Disclosure** ..... 10

# 1. Problem Representation for MapOut

In Week 3, I defined MapOut as an intelligent travel-planning agent and described how it interacts with its environment. The next step is to represent the travel-planning task as a search problem so that the agent can explore different possibilities and generate suitable itineraries.

MapOut aims to help users plan trips based on their interests, budget, and available time. For the initial version, I decided to focus on Ohrid, North Macedonia, and use a small dataset of attractions and activities.

The main problem is to select and organize attractions that match the user's preferences while respecting their budget and time limitations.

In Artificial Intelligence, a search problem can be described using an **initial state, state space, actions, transition model, goal test, and path cost**. These components define how an agent moves from its starting situation toward a solution.

## 1.1 Initial State

The initial state represents the situation before the agent begins planning the itinerary. It contains the information needed to start searching for possible activities. For MapOut, the initial state includes the traveler's starting location, available budget, available time, and selected attractions.

For example, consider a traveler who wants to spend one day in Ohrid with the following preferences:

| Parameter | Destination | Trip Duration | Starting Location | Activity Budget | Interests | Available Time | Selected Attractions |
|---|---|---|---|---|---|---|---|
| **Example Value** | Ohrid | 1 day | Ohrid Center | €50 | History and nature | 6 hours | None |

At the beginning, the traveler has the full budget and available time, and no attractions have been selected. The agent will use this information to explore possible itineraries.

## 1.2 State Representation

A state describes the current situation during the planning process. As the agent selects attractions, the state changes to reflect the remaining resources and activities already included in the itinerary.

For MapOut, I will represent a state as: **S = (L, B, T, V)**

- **L** – Current location.
- **B** – Remaining activity budget.
- **T** – Remaining available time.
- **V** – Ordered list of selected attractions.

For example, the initial state can be represented as: **$S_0$ = (Ohrid Center, 50, 6, [])**

This means the traveler starts in Ohrid Center with €50, 6 hours available, and no selected attractions. After choosing an attraction, the agent updates the location, remaining budget, remaining time, and list of selected attractions. The user's interests remain part of the problem input rather than changing with every state.

## 1.3 State Space

The state space represents all possible states that the agent can reach while planning an itinerary. In MapOut, different combinations and orders of attractions create different possible states. For example, visiting Samuel's Fortress before the Ancient Theatre produces a different itinerary from visiting them in the opposite order.

As more attractions are added to the dataset, the number of possible itineraries increases. For example, with three attractions (A, B, and C), some possible itineraries are:

- A
- B
- C
- A → B
- A → C
- B → A
- A → B → C
- C → B → A

The agent must explore these possibilities while checking whether they satisfy the traveler's constraints. To keep the initial search manageable, I will use a limited number of attractions and avoid visiting the same attraction more than once in a single itinerary.

## 1.4 Actions

Actions represent the decisions the agent can make to move from one state to another. For MapOut, the main action is **selecting the next attraction to include in the itinerary**. Possible actions include visiting Samuel's Fortress, the Ancient Theatre, the Church of St. John at Kaneo, or taking a boat trip on Lake Ohrid.

Each action has associated information, including:

- Estimated activity cost.
- Estimated visiting duration.
- Attraction location.
- Travel time from the current location.
- Attraction category.

An action is allowed only when the attraction has not already been selected and the traveler has enough budget and time to include it. For example, if the traveler has €10 remaining, an activity costing €20 cannot be selected.

## 1.5 Transition Model

The transition model describes how the agent moves from one state to another after performing an action. When MapOut selects an attraction, it updates the traveler's current location, remaining budget, remaining time, and selected attractions.

The new state is calculated by subtracting the attraction's cost from the remaining budget and subtracting both the visiting duration and estimated travel time from the remaining available time.

For example, suppose the traveler starts with €50 and six hours available. For this illustration, assume visiting Samuel's Fortress costs €5 and requires one hour in total, including travel.

**Initial state:** $S_0$ = (Ohrid Center, 50, 6, [])
**Action:** Visit Samuel's Fortress.
**New state:** $S_1$ = (Samuel's Fortress, 45, 5, [Samuel's Fortress])

The traveler now has €45 and five hours remaining, and Samuel's Fortress has been added to the itinerary. The agent continues selecting attractions and updating the state until it reaches a suitable solution or no further feasible activities can be added.

*The prices and durations in this example are assumed values used to explain the transition model, not verified attraction information.*

## 1.6 Goal Test

The goal test determines whether the agent has reached a state that satisfies the requirements of the search problem. For MapOut, a valid itinerary should:

- Include at least two attractions matching the user's interests.
- Stay within the available activity budget.
- Fit within the available time, including estimated travel.
- Avoid visiting the same attraction more than once.
- Respect any availability restrictions included in the dataset.

For the initial search implementation, I plan to define the goal as finding an itinerary containing **at least two attractions that match the user's interests**, without exceeding the available time or budget. If no such itinerary exists, the system should report that the goal cannot be satisfied.

However, finding a valid itinerary does not necessarily mean it is the best possible one. In later versions, I can improve the planning method to compare different itineraries and prioritize those with more relevant attractions and efficient routes.

## 1.7 Path Cost

Path cost represents the total cost of the actions taken from the initial state to the current state. In MapOut, I need to consider two important resources: money and time.

The **total activity cost** is the sum of the estimated prices of all selected attractions. The **total time** includes visiting durations and estimated travel time between locations. For example, an itinerary containing three attractions might require four hours and cost €20.

These values can be calculated as:

**Total Cost = Sum of selected attraction costs**
**Total Time = Sum of visit durations + Sum of travel times**

For the initial search experiments, I can use total time as the path-cost measure while treating the activity budget as a constraint. This will allow me to compare how much time different itineraries require.

Later, I can introduce a more detailed evaluation method that also considers interest relevance and travel efficiency.

# 2. Uninformed Search Algorithms

Uninformed search algorithms explore possible solutions without using additional knowledge about which state is closer to the goal. They rely on the problem definition, including the initial state, available actions, transition model, and goal test.

During Week 4, I will focus on two uninformed search algorithms: **Breadth-First Search (BFS)** and **Depth-First Search (DFS)**. Both algorithms can be used to explore possible attraction combinations, but they follow different search strategies.

## 2.1 Breadth-First Search (BFS)

Breadth-First Search explores the search tree level by level. It first examines states that can be reached using one action, then states requiring two actions, and continues until it finds a goal state. BFS uses a **queue (FIFO – First In, First Out)** to manage the states waiting to be explored.

In MapOut, BFS can explore itineraries with fewer attractions first before moving to combinations containing more attractions.

**Advantages of BFS:**

- It is systematic and explores states level by level.
- It is complete when the branching factor is finite and a solution exists at finite depth.
- It finds a solution with the fewest actions when all actions have equal cost.

**Limitations of BFS:**

- It may require considerable memory because it stores multiple states.
- The number of possible itineraries increases rapidly as more attractions are added.
- Finding the fewest actions does not necessarily produce the cheapest or most relevant itinerary.

BFS can be useful for exploring shorter itineraries and understanding how the search space grows.

## 2.2 Depth-First Search (DFS)

Depth-First Search (DFS) explores one branch of the search tree as deeply as possible before returning to explore other branches. DFS uses a **stack (LIFO – Last In, First Out)** or recursion to manage the search process.

In MapOut, DFS can explore one possible itinerary by adding attractions until it reaches a solution or cannot continue. If a path does not satisfy the requirements, the algorithm returns to an earlier state and tries another option. This process is called **backtracking**.

**Advantages of DFS:**

- It generally requires less memory than BFS.
- It can explore a complete itinerary before considering alternative combinations.
- It is relatively simple to implement using recursion.

**Limitations of DFS:**

- It does not guarantee finding the shortest or least-cost solution.
- It may explore unsuitable branches before finding a valid itinerary.
- In an infinite-depth search space, it may fail to terminate without additional restrictions.

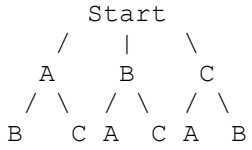
For MapOut, DFS can be useful for exploring attraction combinations and testing whether complete itineraries satisfy the traveler's constraints.

## 2.3 Example of BFS and DFS in MapOut

To better understand how BFS and DFS work, consider a traveler who wants to visit two attractions in Ohrid and has enough time and budget for both. The available attractions are:

- A – Samuel's Fortress
- B – Ancient Theatre
- C – Church of St. John at Kaneo

The possible itineraries can be represented using a search tree:




Each branch represents selecting another attraction. For example, **Start → A → B** means visiting Samuel's Fortress first, followed by the Ancient Theatre.

**BFS** explores the tree level by level. It first checks each attraction separately (A, B, C), then moves to combinations of two attractions (A → B, A → C, B → A, etc.). Since the goal is to visit two attractions, BFS stops when it finds the first combination that satisfies the requirements.

**DFS** explores one path as deeply as possible before trying another. For example, it starts with A and continues to B, creating the itinerary A → B. If this combination does not satisfy the requirements, DFS goes back and tries another option, such as A → C. This process is called *backtracking*.

The main difference is that **BFS explores all possibilities at one level before moving deeper, while DFS follows one path before exploring alternatives**. Both can find valid itineraries, but they may find different solutions depending on the order of exploration. For this example, all shown combinations are assumed to fit within the traveler's budget and time.

## 2.4 Search Strategy for MapOut

For the initial version, I plan to test both BFS and DFS using a small dataset of attractions in Ohrid. This will help me understand how the algorithms explore different combinations and find possible itineraries.

Both algorithms will use the same starting information, including the traveler's budget, available time, and interests. They will also follow the same rules, such as avoiding repeated attractions and staying within the available budget and time.

I will compare **how many states each algorithm explores, whether it finds a valid itinerary, and how long the search takes**. This will help me understand their differences and limitations.

Since BFS and DFS do not use additional information to decide which attractions are more suitable, they may not always produce the best itinerary for the traveler.

In **Week 5**, I plan to explore informed search algorithms and heuristics to improve MapOut's recommendations by considering attraction relevance and travel distance.

# 3. Data and Environment Preparation

The Week 4 milestone also requires preparing the first version of the project's data or environment. For MapOut, this means creating a small dataset containing attractions and activities in Ohrid.

The dataset will provide the information needed for the agent to evaluate available activities and perform search operations.

## 3.1 Initial Dataset Structure

I stored the attraction information in a CSV file named ***attractions.csv***.

Each attraction will contain the following attributes:

| Attribute | Description |
|---|---|
| **ID** | Unique identifier for each attraction |
| **Name** | Name of the attraction or activity |
| **Category** | History, Adventure, Relaxation, Food, Entertainment |
| **Latitude** | Geographical latitude |
| **Longitude** | Geographical longitude |
| **Estimated Cost** | Approximate activity price in euros |
| **Visit Duration** | Estimated time required to visit the attraction |
| **Availability** | Seasonal or general availability information |

The initial dataset contains 15 attractions and activities in Ohrid and the surrounding area. Examples include Samuel's Fortress, the Ancient Theatre, the Church of St. John at Kaneo, Bay of Bones Museum, and the Monastery of Saint Naum.

## 3.2 Why I Chose a Small Dataset

I decided to begin with a limited number of attractions because the main objective is to understand and test how search algorithms work. A smaller dataset will make it easier to check whether the agent correctly updates states, respects constraints, and avoids repeating attractions.

It will also help me identify problems in the search process before adding more locations and activities. Once the basic search functionality works, I can expand the dataset and test how the algorithms perform with more possible combinations.

## 3.3 Data Quality and Limitations

One challenge is that attraction information may change over time. Ticket prices, opening hours, and seasonal activities may not always remain the same. For the initial dataset, I used estimated values where appropriate and left unknown costs empty instead of assuming they were free.

I also included sources for the attraction information so that the dataset can be reviewed and updated. Some information still requires verification. The initial version does not depend on real-time weather or attraction availability, but these features may be considered in later versions.

# CONCLUSION

During Week 4, I focused on representing MapOut as a search problem and understanding how search algorithms can be used to generate travel itineraries. I defined the initial state, state space, actions, transition model, goal test, and path cost. I also explored BFS and DFS and explained how they can search through different combinations of attractions in Ohrid.

For the practical part, I created a dataset of 15 attractions and developed a Python program to load, validate, and display the data. This prepares the project for implementing search algorithms in the following weeks.

# AI USE DISCLOSURE

I used ChatGPT (OpenAI) to clarify concepts, review examples, research attraction information, and improve my documentation. I also used it for guidance while developing my Python code. I reviewed the suggestions, implemented and tested the code myself, and made the final decisions about the project.